Azizbek Arzikulov

Backend / Full-Stack Engineer

Product systems • Integrations • Deployment • Debugging

What you'll see

- 4 flagship systems with evidence-first case studies
- Architecture and delivery decisions (not just UI screenshots)
- Role-fit mapping directly to Retentics requirements
- Links to live demos + repositories

I am a full-stack engineer who tries to solve problems firsthand rather than just writing code.

GitHub: github.com/azizbekdevuz

Trainium: trainium.shop

GitGuard: gitguardian.online

Positioning

Truth, framed like a professional.

Core value

I build and ship systems end-to-end — backend services, data models, auth, integrations, real-time flows, and deployment — and I’m the person who closes the “integration gap” when separate modules don’t actually work together.

What makes me useful in a fast-moving position

- Evidence-first engineering: reproduce → isolate → fix → harden
- Comfortable owning delivery beyond code: release, ops, reliability
- Clear English communication (technical + product)
- Strong integration skills: BFF wiring, auth/session fixes, API contracts

How I work

A repeatable delivery loop — not “build UI, hope it works”.

Scope

Requirements → constraints → risks

→

Design

Data models → API boundaries → auth

→

Build

Incremental delivery + tests where they matter

→

Integrate

Contracts, BFF, sessions, edge cases

→

Deploy

CI, envs, SSL/HTTPS, runbooks

→

Harden

Logs, monitoring, performance, refactors

This is what I’m optimized for

Retentics needs reliability + throughput while the product is evolving. My strength is turning moving requirements into stable systems — and cleaning up integration debt fast.

Case Study — Trainium

Live: https://trainium.shop

Production-minded e-commerce + admin platform (solo, end-to-end).

What it proves

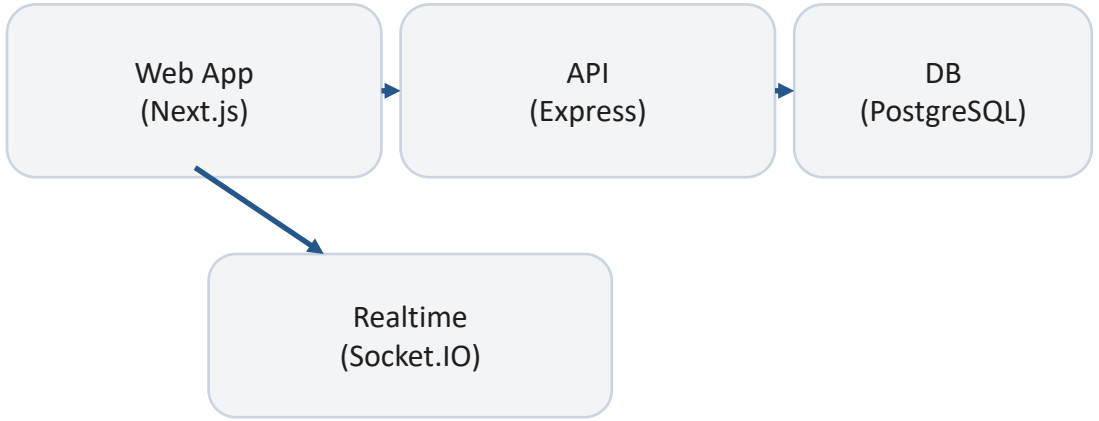
- Real product scope: storefront + admin workflows
- Auth + roles, payments, order flows
- Relational data modeling (Postgres/Prisma)
- Real-time notifications, i18n
- Deployment/ops ownership (VPS, Docker, Nginx, SSL)

GitHub: https://github.com/azizbekdevuz/trainium

Stack

Next.js • Express • PostgreSQL • Prisma • Socket.IO • Docker • Nginx

Architecture (simplified)



Key decisions

- Clean domain models and safe migrations
- Role-aware admin boundaries
- Ops-first deploy: SSL/HTTPS, env separation, hardening

Live: trainium.shop

Repo: github.com/azizbekdevuz/trainium

Case Study — GitGuard Agent

Live: https://gitguardian.online

AI-guided Git recovery (CLI + web) with reversible-by-design workflows.

What it proves

- System design across CLI → backend → UI
- Safe workflows: every action has undo paths
- Python agent service integration
- Productized developer tooling
- Deployment discipline + docs

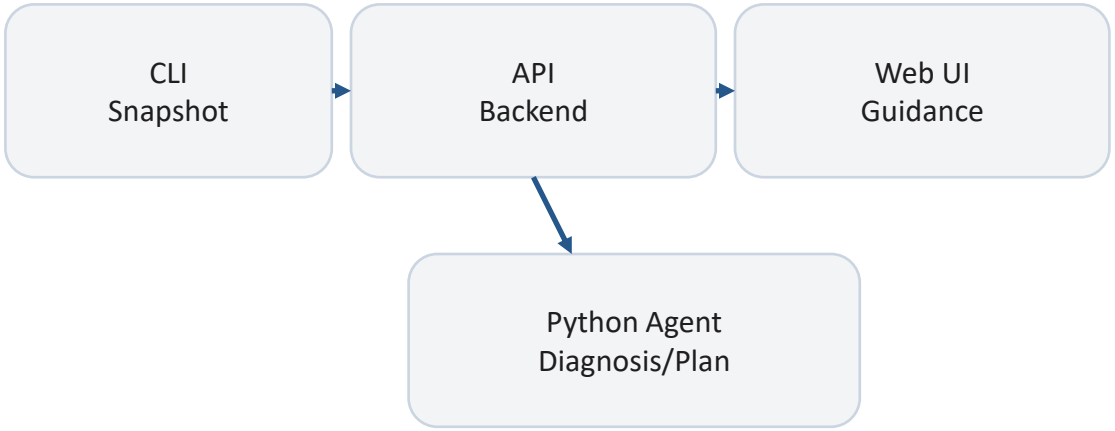
GitHub: https://github.com/azizbekdevuz/gitguardian

NPM package: https://github.com/users/azizbekdevuz/packages/npm/package/gitguard-cli

Stack

Web + CLI • Python agent service • DB layer • Deployment docs

Architecture (simplified)



Agent + service integration, clear workflows, and “safety-first” thinking map well to production backend reliability.

Live: gitguardian.online

Repo: github.com/azizbekdevuz/gitguardian

Case Study — Rumi AI

Multilingual AI companion (RAG + LLM). Frontend + integration ownership.

What I owned

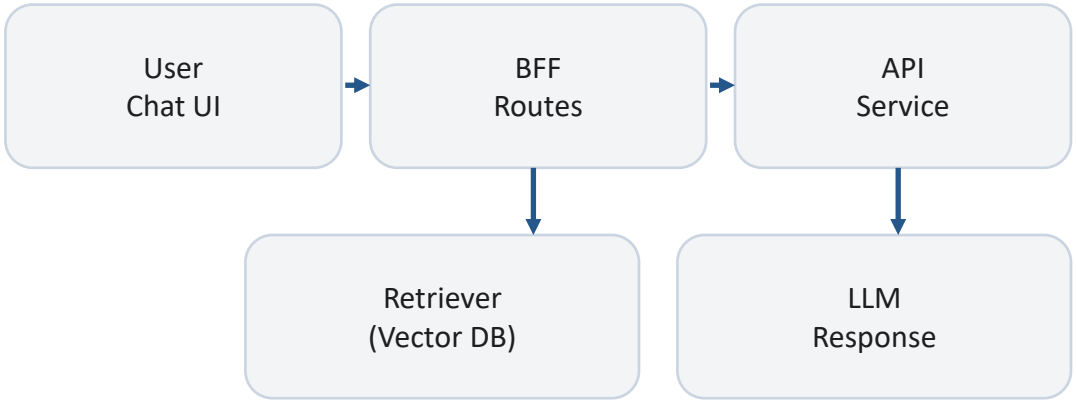
- Multilingual UI (incl. RTL Persian) and UX delivery
- BFF-style integration: wiring backend capabilities into product flow
- Auth/session + API contract alignment
- Reliability improvements: streaming, edge cases, guardrails

GitHub: https://github.com/azizbekdevuz/rumi-ai

Why it matters for Retentics

This is integration-heavy product engineering: making separate services behave like one coherent system.

RAG pipeline (concept)



Key focus: streaming + multilingual UX + session reliability

Repo: github.com/azizbekdevuz/rumi-ai

Case Study — PatchPilot

Bug recording → structured report → tests → patch suggestions (hackathon system, real orchestration).

What it proves

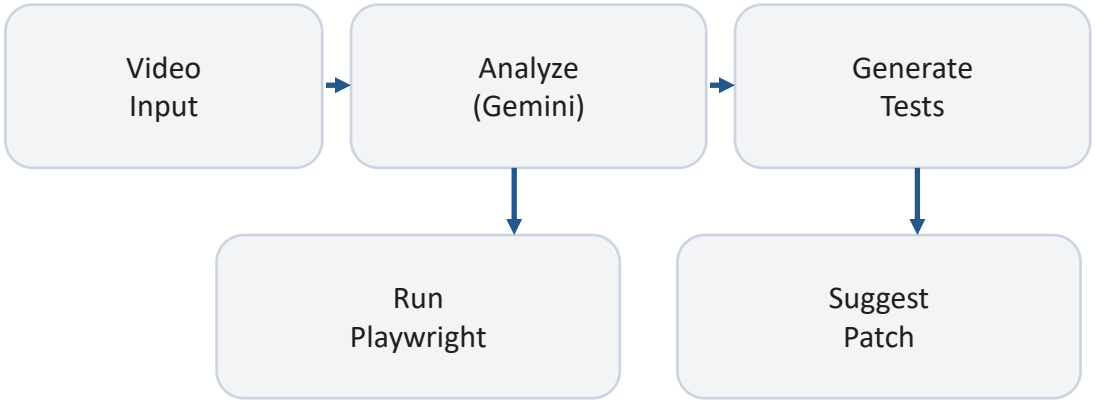
- Multi-step workflow orchestration (not a prompt wrapper)
- Verification loop via Playwright execution
- Backend responsibilities: state, retries, errors, health
- Production-minded integration + deployment

GitHub: https://github.com/Prithwis-2023/patchpilot

Stack

FastAPI • Next.js • Playwright • Orchestrated multi-call pipeline

Workflow (simplified)



Key: verification via test execution (reduces hallucinated fixes)

Repo: github.com/Prithwis-2023/patchpilot

Demo: patchpilot-frontend-beta.vercel.app

Additional backend systems.

Not featured case studies, but they strengthen backend breadth.

Nestar

- NestJS + GraphQL
- JWT auth + roles
- Mongoose/NoSQL patterns
- Jest + e2e structure

Repo link

Burak

- Express + TypeScript
- Sessions/JWT
- Helmet + rate limiting
- Socket.IO

Private repo → code review available upon request

Leo Academy

- LMS-style backend
- Stripe (test mode)
- Uploads + media pipeline
- Email + session flows

Private repo → code review available upon request

Requirements mapping

Direct mapping from requirements → evidence.

Requirement	Evidence in my work
Python-based development	Rumi AI (integration), PatchPilot (FastAPI), GitGuard (agent service)
Docker / container environments	Trainium deployment; VPS ops patterns; project-level containerization
Relational + NoSQL databases	Trainium (PostgreSQL/Prisma), Nestar/Burak/Leo (MongoDB/Mongoose)
CI/CD	GitHub Actions CI on shipped projects (e.g., Trainium)
Service stability + performance mindset	Integration fixes, reliability hardening, “prove it” debugging loop
English collaboration	Research lab + cross-team integration work; documentation-first projects
AWS / Serverless	Gap: currently limited hands-on; ramping plan available on request

Note: I’m not trying to “sound senior.” I’m showing evidence and ownership. That’s the bet.

Contact

Email azizbek.dev.ac@gmail.com

GitHub github.com/azizbekdevuz

Key projects trainium.shop gitguardian.online

rumi-ai repo patchpilot repo

Location Seoul, South Korea